

EXPERIMENT 9 -A* ALGORITHM

AIM

To develop a Python program to implement the A* algorithm for finding the shortest path between a source node and a goal node.

OBJECTIVE

To demonstrate how the A* search algorithm finds the optimal path using the cost function $f(n) = g(n) + h(n)$.

ALGORITHM

- Start.
- Initialize the Open List with the start node and create an empty Closed List to store the visited nodes.
- Select the node from the Open List that has the lowest evaluation function, $f(n) = g(n) + h(n)$, where $g(n)$ is the path cost and $h(n)$ is the heuristic estimate.
- Expand the selected node, calculate the cost of its neighboring nodes, and update the Open List if a better path is found.
- Repeat the process until the goal node is reached or the Open List becomes empty (indicating that no path exists).
- Display the shortest path from the start node to the goal node and Stop.

SAMPLE CODE

```
graph = {  
    'A': [('B', 1), ('C', 3)],  
    'B': [('D', 3), ('E', 6)],  
    'C': [('F', 5)],  
    'D': [],  
    'E': [('G', 2)],  
    'F': [('G', 2)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7,  
    'B': 6,  
    'C': 4,  
    'D': 3,  
    'E': 2,  
    'F': 1,  
    'G': 0  
}
```

```
def a_star(start, goal):  
    open_list = [(start, 0)]  
    visited = set()  
  
    while open_list:  
        open_list.sort(key=lambda x: x[1] + heuristic[x[0]])  
        node, cost = open_list.pop(0)  
  
        if node == goal:
```

```
print("Goal Reached:", goal)
```

```
return
```

```
if node not in visited:
```

```
    print("Visited:", node)
```

```
    visited.add(node)
```

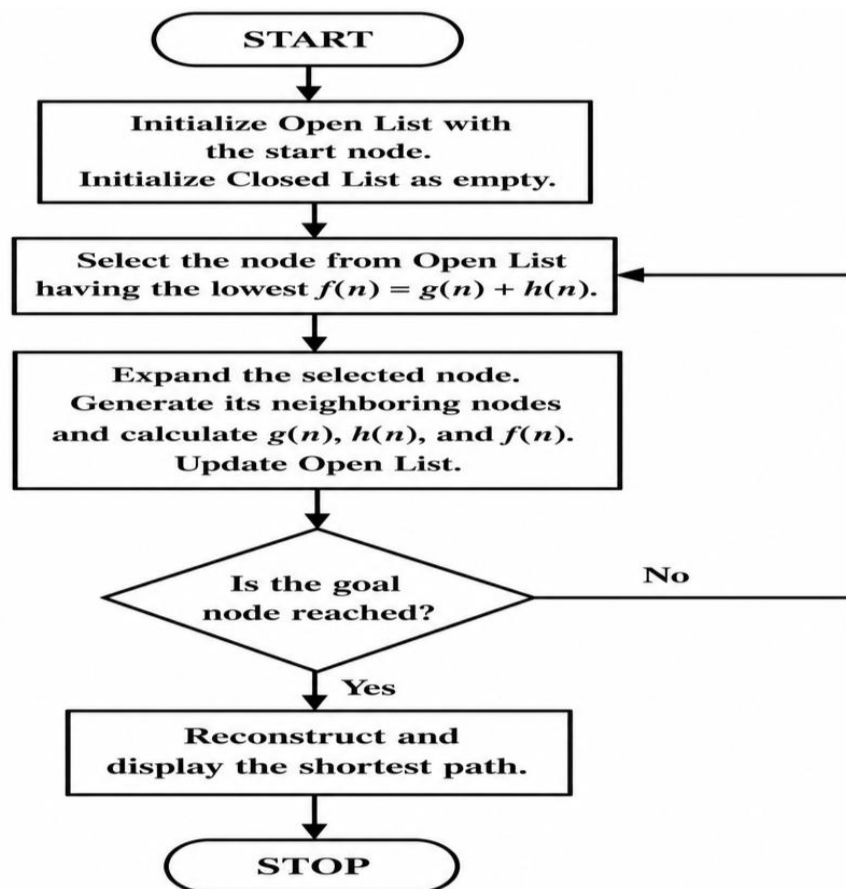
```
for neighbour, weight in graph[node]:
```

```
    if neighbour not in visited:
```

```
        open_list.append((neighbour, cost + weight))
```

```
a_star('A', 'G')
```

FLOW CHART



OUTPUT

```
Visited: A  
Visited: B  
Visited: C  
Visited: D  
Visited: E  
Visited: F  
Goal Reached: G  
  
Process finished with exit code 0
```

RESULT

The Python program was executed successfully. The vacuum cleaner agent sensed the environment, cleaned all the dirty rooms, and displayed the final status of each room.